LANCE, a Network-Native LLM Agent and Benchmark for Multi-Hop IoT Penetration Testing

Anonymous Authors
Paper #XXXX, Anonymized for Double-Blind Review

***Abstract*—Autonomous LLM penetration-testing agents are typically evaluated on single hosts, yet IoT deployments expose risk through network reachability: gateways, brokers, cameras, and OT services form attack paths whose security depends on pivots across devices. We present IoTChainBench, a reproducible network-scale IoT benchmark with 15 Proxmox/Ansible scenarios, 145 devices in total (133 in vulnerability-injected scenarios), and 229 injected vulnerabilities mapped to OWASP IoT Top 10 and MITRE ATT&CK for ICS. We also introduce LANCE, a six-phase network-native harness that combines topology priors, active discovery, per-device triage, per-vulnerability verification, intrusion attempts, and report generation. The benchmark adds two metrics absent from single-host evaluations: evidence-level coverage (L1 detection, L2 exploitation, L3 exfiltration) and Multi-Hop Reach (MHR_k), which measures recall on vulnerabilities that require at least k pivots or zone transitions. We describe the benchmark, evaluator, and experimental protocol for isolating architectural effects independently from model choice. On the 10 main scenarios, LANCE achieves 0.962 recall, 0.922 precision, 0.940 F1, and 86.1% CVSS-weighted score. On external single-host corpora, a blind AutoPenBench run captures 7/33 flags (21.2%), while an informed run captures 9/33 (27.3%); a 191-case informed Vulhub run gives a conservative 39–48% useful-finding lower bound, and a separate 328-case blind run yields 29%.**

***Index Terms*—IoT security, penetration testing, large language models, autonomous agents, benchmark, cyber-physical systems**

1. Introduction

IoT deployments are compromised through paths, not just devices. Mirai [26], Verkada [27], and Stuxnet [28] each demonstrate this: impact came from chained access across cameras, brokers, PLCs, and segmented boundaries.

Yet LLM penetration-testing agents are evaluated on a single host at a time: isolated containers, VMs, or CTF services scored independently. This gap is methodological, not merely architectural. Single-host corpora cannot measure three capabilities central to IoT/OT security: finding vulnerabilities behind pivots, reasoning over domain protocols (MQTT, Modbus, CoAP, LoRaWAN), and distinguishing banner-level detection from confirmed exploitation or exfiltration. Without a network-scale benchmark, strong single-host scores overstate readiness for real deployments.

Contributions. This paper makes four contributions.

- **IoTChainBench**: a reproducible network-scale benchmark with 15 IoT/OT scenarios, 145 devices, and 229 injected vulnerabilities mapped to OWASP IoT Top 10 and MITRE ATT&CK for ICS, deployed via Ansible with per-vulnerability ground truth.
- **LANCE**: a six-phase network-native harness (topology $\rightarrow$ discovery $\rightarrow$ triage $\rightarrow$ verification $\rightarrow$ intrusion $\rightarrow$ report) achieving 0.940 F1 and 86.1% CVSS-weighted score on the 10 main scenarios.
- **Evidence-aware metrics**: L1/L2/L3 scoring (detection, exploitation, exfiltration) and MHR_k (recall as a function of firewall-hop depth), both absent from existing flag-capture benchmarks.
- **Controlled evaluation**: comparison against adapted LLM-agent baselines (CAI, PentestGPT, VulnBot) under a fixed model and budget, with transfer validation on AutoPenBench and Vulhub.

All artifacts are released at `ANONYMIZED_URL`.

2. Related Work

This section reviews prior work relevant to our approach. We first survey LLM-driven penetration-testing agents and the benchmarks used to evaluate them, then discuss the classical attack-graph literature that informs LANCE's topology-aware design, and finally review IoT firmware analysis and the security frameworks underlying our scenario taxonomy.

2.1. LLM Penetration-Testing Agents

Single-agent systems. Happe and Cito [1] were among the first to show that a single LLM driving a shell can autonomously perform privilege escalation on Linux VMs. PENTESTGPT [3] improved this with a three-module design over a *Pentesting Task Tree*, outperforming direct GPT-3.5 and GPT-4 usage on HackTheBox and VulnHub targets. HACKINGBUDDYGPT [2], AUTOATTACKER [4], and NEBULA [5] extend this line as open-source tools.

Multi-agent pipelines. VULNBOT [6] decomposes pentesting into reconnaissance, scanning, and exploitation over a

Penetration Task Graph, gaining 21 points end-to-end over its single-agent baseline (30.3% on AutoPenBench with Llama 3.1-405B). AUTOPENTESTER [7] adds RAG across five agents for a +27% subtask gain over PENTESTGPT on HackTheBox. PENTESTAGENT [8] and CAI [9] generalise this blueprint with modular tool use and a public leaderboard.

Model-level improvements. PENTEST-R1 [10] applies two-stage reinforcement learning on reasoning traces. XOFFENSE [11] fine-tunes Qwen3-32B on Chain-of-Thought penetration testing data, reaching 72.72% task completion and 79.17% best-of-five subtask completion on AutoPenBench. CHECKMATE [16] couples an LLM with a *Classical Planning+* module, gaining more than 20% on Vulhub over a Claude Code + Sonnet 4.5 baseline.

Common limitation. Despite this diversity, reported evaluations remain single-target or challenge-level. They do not score subnet reachability, pivot depth, or per-vulnerability progress across IoT/OT protocols (MQTT, Modbus, LoRaWAN, CoAP), which is the gap IOTCHAINBENCH and LANCE address.

2.2. LLM Cybersecurity Benchmarks

Challenge-level corpora. AUTOPENBENCH [18] is the dominant evaluation substrate: 33 Docker-container tasks with milestone and flag-based scoring, used by VULNBOT, PENTEST-R1, and XOFFENSE. VULHUB [25] provides reproducible vulnerable environments used by CHECKMATE. CAIBENCH [22], NYU CTF BENCH [19], and CYBENCH [20] extend coverage to CTFs, attack-and-defense ranges, and knowledge tests. These corpora score at challenge or flag level. They do not provide per-vulnerability ground truth tied to reachability, hop depth, or IoT/OT protocols.

Multi-host labs. LUDUS [23] and GOAD [24] deliver Proxmox+Ansible deployments of enterprise Active Directory networks. They are labs and training ranges, not scored LLM benchmarks. Both model multi-host compromise and lateral movement, but neither ships a per-vulnerability ground-truth schema, hop-depth annotations, or an evaluator. Both also target enterprise AD rather than IoT/OT protocols. To our knowledge, no prior benchmark combines multi-host reachability, IoT/OT protocol coverage, per-vulnerability ground truth, and evidence-level scoring beyond binary flag capture (Table 1).

2.3. Attack Graphs and Network Reachability

Representing a network as a graph for security analysis predates deep learning by decades. Phillips and Swiler [29] introduced graph-based network-vulnerability analysis. Sheyner *et al.* [30] developed automated attack-graph generation via model checking, and MULVAL [31] cast the problem as Datalog inference. These systems compute reachable attacker states from a *static* encoding of host configurations and exposed services. LANCE borrows this reachability abstraction but operates on a *live* network: edges are validated by probes, the LLM proposes the next action, and the graph is updated as hosts are discovered. Classical attack-graph generation is therefore complementary rather than competing.

2.4. IoT Security Context

Scenario design and ground-truth taxonomy follow three references: OWASP IoT Top 10 [32] for vulnerability categories, MITRE ATT&CK for ICS [33] for OT-tactic annotations, and NIST SP 800-82 Rev. 3 [34] for OT security and segmentation conventions.

Empirical IoT-security artefacts fall into two families. **Network-traffic datasets** (IoT-23 [35], TON_IoT [36], Edge-IIoTset [37]) provide labeled traces for intrusion detection. They are not deployable pentest targets with per-vulnerability ground truth or controllable reachability. **Firmware-focused work** (Costin *et al.* [38], FIRMADYNE [39], FIRMAE [40], IOTGOAT [41]) exercises single-device firmware analysis. These datasets measure detection or firmware-analysis capability. IOTCHAINBENCH instead measures active security assessment on deployed multi-device networks with firewall-enforced reachability.

TABLE 1. GAP ANALYSIS. *Scale*: SINGLE HOST VS. MULTI-DEVICE NETWORK. *IoT/OT*: APPLICATION-LAYER IOT/OT PROTOCOLS ON THE TEST SURFACE. *GT*: GROUND-TRUTH GRANULARITY (FLAG, TASK-STEP, PER-VULNERABILITY, NONE). *MH*: MULTI-HOP SCORING.

System	Scale	IoT/OT	GT	MH
LLM pentest agents				
Happe & Cito [1]	1 host	✗	flag	✗
PentestGPT [3]	1 host	✗	task-step	✗
VulnBot [6]	1 host	✗	task-step	✗
AutoPentester [7]	1 host	✗	task-step	✗
CAI [9]	1 host	✗	flag	✗
Pentest-R1 [10]	1 host	✗	task-step	✗
xOffense [11]	1 host	✗	task-step	✗
EnIGMA [12]	1 host	✗	flag	✗
CheckMate [16]	1 host	✗	task-step	✗
Evaluation corpora				
AutoPenBench [18]	1 host	✗	flag	✗
CAIBench / RCTF2 [22]	1 host	partial	flag	✗
Vulhub [25]	1 host	✗	flag	✗
Ludus + GOAD [23], [24]	network	✗	none	✗
IoTChainBench (ours)	**network**	✓	**per-vuln**	✓

3. Threat Model and Problem Definition

Problem statement. Given a target subnet with a designated entry point e declared in each scenario's topology metadata, *network-scale penetration testing* is the task of discovering all hosts reachable from e, identifying their vulnerabilities, producing evidence of exploitability, and reporting findings across the full network. The unit of evaluation is the network, not an individual host: IoT risk is driven by pivot chains (e.g., entry point → gateway → MQTT broker →

OT controller), and single-host metrics are blind to vulnerabilities that only become reachable after one or more pivots.

Attacker model. The attacker is an external adversary with Layer-3 access to the target subnet but no prior credentials, no out-of-band knowledge of deployed software, and no privileged vantage beyond active scanning. Their objective is data exfiltration (sensor readings, credentials) or OT disruption (reaching industrial-control services behind a segmentation boundary). They may use standard pentest tooling (Nmap, MQTT clients, SSH, HTTP) augmented by an LLM with tool-calling, within a bounded turn budget (§5.2). This adversarial setting requires multi-step reasoning across heterogeneous protocols, which motivates the use of an LLM-driven agent rather than a static scanner.

Evidence levels. A finding is solved at the deepest level the agent can demonstrate within its turn budget:

- `L1` *detection*: a port scan or banner confirms the issue (e.g., unauthenticated MQTT port `1883` open).
- `L2` *exploitation*: an active interaction demonstrates it (e.g., anonymous MQTT subscribe on `#`, SSH login with default credentials).
- `L3` *exfiltration*: sensitive content is retrieved (e.g., MariaDB `root` dump, camera RTSP stream).

This ordering governs both LANCE Phase 4 prompting (§5.1) and evaluator scoring (§4.5).

Scope. We evaluate at the IP/application layer on virtual IoT networks. Out of scope: physical and side-channel attacks, RF-layer exploitation, firmware extraction, post-exploitation persistence, and insider threats.

4. IoTChainBench: A Network-Scale Benchmark

4.1. Design Principles

IoTChainBench is built around six principles:

- **Reproducibility.** Full infrastructure deploys from Ansible playbooks against a Proxmox cluster; vulnerabilities are injected idempotently with no manual steps.
- **Architectural diversity.** Scenarios span five network patterns with reachability enforced by OpenWrt firewall rules in segmented/VLAN cases.
- **Protocol coverage.** Each scenario mixes IoT and OT protocols (MQTT, Modbus, LoRaWAN, CoAP, SNMP, SSH, HTTP, FTP, Telnet) to exercise protocol-specific reasoning.
- **Ground-truth granularity.** Every vulnerability is documented in YAML with device, IP, severity, OWASP IoT and MITRE ICS mappings, optional CVE, indicators, and a verification command.
- **Scalability axis.** Ten main scenarios (4–8 devices) and three scalability scenarios (15–35 devices) separate pattern coverage from network size.
- **Hardened controls.** Two vulnerability-free topologies reusing `s_1` and `s_4` measure agent False Positive rate.

4.2. Network Topologies and Scenario Catalogue

IoTChainBench defines five topological patterns that vary the number of pivots required to reach the deepest vulnerable service, directly setting the MHR_k difficulty axis:
nosep

1) **P1 — Flat.** Single subnet; all devices directly reachable from the entry point.
2) **P2 — Gateway-centric.** One IoT gateway mediates all back-end access; back-end services unreachable without gateway compromise.
3) **P3 — Segmented IT/OT.** Admin/IT/OT zones with ACLs; the OT zone (Modbus PLCs, HMI) requires a jump-host pivot.
4) **P4 — Hub-centric star.** A central hub is the sole path to the MQTT broker, DB, and camera; peripherals are mutually isolated.
5) **P5 — Edge-cloud pivot.** Edge and cloud subnets are separated; only the edge gateway bridges them.

The 15 scenarios (Table 2) are organised along three axes: 10 *main* scenarios instantiate the patterns above at 4–8 devices; 3 *scalability* stressors push to 15, 17, and 35 devices using logical multi-zone and real-VLAN variants; and 2 *hardened controls* replicate the topologies of s_1 and s_4 with no injected vulnerability, used to measure agent false-positive rate.

TABLE 2. THE 15 IOTCHAINBENCH SCENARIOS: 13 VULNERABILITY-INJECTED (133 DEVICES, 229 VULNERABILITIES), PLUS 2 HARDENED CONTROLS (12 DEVICES, 0 VULNERABILITIES).

ID	Pattern / role	Dev.	Vulns	Theme
Main scenarios (pattern coverage)				
`s_1`	P1 Flat	4	12	Minimal IoT (MQTT/Web/SSH)
`s_2`	P1 Flat	6	13	Flat IoT + IT mix
`s_3`	P2 Gateway-centric	8	18	NATO lab replica
`s_4`	P3 Segmented IT/OT	8	18	Industrial (Modbus/HMI)
`s_5`	P4 Hub-centric star	8	15	Smart building (cameras/NVR)
`s_6`	P4 Hub-centric star	6	16	Home automation hub
`s_7`	P5 Edge-cloud pivot	6	14	Edge → cloud API
`s_8`	P3 Segmented IT/OT	8	14	IT/IoT/OT 3-zone industrial
`s_9`	P1 Flat (mesh)	6	11	Mesh IoT smart-city outdoor
`s_10`	P1 Flat (variants)	6	13	Role-variant stress
Scalability stressors				
`s_11`	Smart City (flat)	15	23	3 zones same /24 (no isolation)
`s_13`	VLAN-segmented	17	20	Real Proxmox VLAN bridge (3 VLANs)
`s_12`	Smart City (large)	35	42	Largest network (34 LXC services)
Total		**133**	**229**	

4.3. Deterministic Vulnerability Injection

All vulnerabilities are injected by a dedicated Ansible role parameterized by `scenario_id`, ensuring fully reproducible and idempotent deployments. Each scenario YAML

declares router-level weaknesses (Telnet, WAN admin, FTP) and per-service role definitions that trigger targeted injections, covering the most common real-world IoT misconfigurations: anonymous MQTT access, weak SSH credentials, and passwordless database roots. Every injected vulnerability is mapped to OWASP IoT Top 10 and MITRE ATT&CK for ICS, providing a structured taxonomy for evaluation.

The 229 injected vulnerabilities span 11 categories (Table 3), dominated by misconfigurations (68), missing authentication (41), and data exposure (38). Severity skews high: 107 HIGH and 37 CRITICAL instances, reflecting realistic IoT deployment weaknesses rather than theoretical edge cases.

TABLE 3. VULNERABILITY CATEGORY AND SEVERITY DISTRIBUTION (229 INSTANCES ACROSS 13 SCENARIOS).

Category	Count	CRIT	HIGH	MED	LOW
`misconfiguration`	68	7	36	25	0
`no_authentication`	41	17	21	3	0
`data_exposure`	38	0	14	24	0
`default_credentials`	29	3	26	0	0
`info_disclosure`	13	0	0	0	13
`code_injection`	11	10	1	0	0
`cve`	9	0	3	6	0
`insecure_update`	9	0	4	5	0
`weak_crypto`	6	0	0	1	5
`missing_header`	3	0	0	0	3
`privilege_escalation`	2	0	2	0	0
Total	**229**	37	107	64	21

4.4. Ground-Truth Schema

Each scenario ships a YAML file with one entry per injected vulnerability. Each entry records the affected device, IP, role, severity, category, OWASP IoT Top 10 and MITRE ATT&CK for ICS mappings, an optional CVE, expected detection indicators, and a verification command that confirms exploitability. This per-vulnerability granularity, absent from flag-capture benchmarks, enables the evidence-level and MHR_k metrics defined in §4.5.

```
- id: V1
  title: "MQTT without authentication"
  severity: high
  category: misconfiguration
  owasp_iot: "I1 / I9"
  mitre_ics: "Initial Access, Collection"
  indicators:
    - "allow_anonymous true"
  verification: "mosquitto_sub -h 192.168.100.11 -t '#' -v"
```

Figure 1. Ground-truth entry example (fields trimmed for brevity).

Scenarios may additionally declare a `bonus_types` list of vulnerability types adjacent to the injected set. Agent findings that match these types are counted as neither True Positives nor False Positives, avoiding penalties for legitimate hardening recommendations while keeping the recall denominator fixed.

4.5. Scoring and Evaluation Metrics

Finding matcher. Each LLM finding is matched against ground truth in three ordered passes: (1) exact CVE match; (2) `(ip, vuln_type)` with canonicalised type; (3) `(ip, severity)` fallback. Unmatched findings are False Positives. Duplicate findings on `(ip, type, port)` are resolved by retaining the lower severity to counteract inflation observed in pilot runs.

Weighted score. The composite score is a severity-weighted True-Positive sum (w: critical 4, high 3, medium 2, low 1) normalized by $\sum_{g \in \text{GT}} w(g)$, with $0.5\times$ and $0.75\times$ penalties for fallback and severity-mismatch matches respectively.

Evidence levels. Per True Positive, the evaluator tracks the deepest level achieved: L1 detection, L2 exploitation, L3 exfiltration.

$$\text{L}k\text{-coverage} = \frac{|\{t \in \text{TP} : \text{level}(t) \geq k\}|}{|\text{TP}|}$$

This metric is absent from binary flag-capture benchmarks.

Multi-Hop Reach. For each ground-truth vulnerability g, hop depth counts the minimum number of pivots from entry point e:

$$\text{hop\_depth}(g) = \min_{p \,\in\, \text{paths}(e \to d_g)} \max(0,\, |p|_E - 1)$$

Multi-Hop Reach at depth k restricts recall to deep findings:

$$\text{MHR}_k = \frac{|\{g \in \text{GT} : \text{hop\_depth}(g) \geq k \,\wedge\, g \in \text{TP}\}|}{|\{g \in \text{GT} : \text{hop\_depth}(g) \geq k\}|}$$

MHR_1, MHR_2, MHR_3 measure recall at increasing pivot depth. Agents limited to directly exposed hosts score zero for $k \geq 1$.

5. LANCE: A Network-Native LLM Pentest Harness

Existing LLM pentest harnesses treat the network as a flat list of targets, letting a single agent accumulate unbounded context as the network grows. LANCE instead decomposes network pentesting into six phases with typed deliverables. The key design decision is *parallelization at the device and vulnerability granularity*: rather than a single monolithic agent holding the full network state, we spawn dedicated micro-agents per target with constrained tool sets and aggregate deterministically. This keeps each micro-agent's context bounded at $O(1)$ in the number of network devices, with cross-device state re-introduced only in Phase 5 (intrusion) and Phase 6 (report). Vulnerability type strings produced by LLM agents are resolved through a canonical taxonomy of 11 categories (Table 3), filtering noise and deduplicating collisions before evaluation.

5.1. Pipeline

Phase 1: Topology prior. Phase 1 loads a directed graph $G = (V, E)$ of the target network, where V are devices and E are protocol-labeled links. In *scenario mode*, the topology is loaded from the benchmark YAML; in *discovery mode*, the graph starts empty and is populated by Phase 2. The output enumerates the attack surface and ranks pivot candidates by betweenness centrality (identifying devices whose compromise enables the most lateral paths).

Phase 2: Network discovery. Phase 2 executes active reconnaissance across the target `/24` using host discovery, service/version scanning, L2 vendor probing, and protocol-specific probes (MQTT, SSH, HTTP). Unlike single-host harnesses, the agent derives the target IP from the CIDR scope rather than receiving it directly.

Phase 3: Per-device parallel triage. Phase 3 runs a deterministic scanner pass per device, then spawns one LLM micro-agent per device with only that device's scan output and a minimal tool set (`http_get`, `cve_search`). Per-device outputs are merged and deduplicated via severity-aware rules. This decomposition keeps each agent's context constant regardless of network size, at the cost of cross-device correlation, which Phase 5 partially recovers through lateral movement.

Phase 4: Per-vulnerability verification. Phase 4 spawns one micro-agent per Phase 3 finding with full operational tool access (`ssh_login`, `mysql_query`, `mqtt_listen`, `modbus_scan`, `telnet_connect`). Each agent attempts the deepest evidence level achievable (L1–L3, §3) within a bounded turn budget. Phase 3 confirmed findings override Phase 4 errors to avoid erasing directly-observed indicators.

Phase 5: Intrusion and lateral movement. Phase 5 turns the per-device vulnerability inventory into a network-scoped infiltration campaign. The intrusion agent attempts credential spraying, lateral movement along edges present in the topology graph from Phase 1 (reflecting observed or asserted reachability), and crown-jewel access on back-end services unreachable from the entry point. Every hop is recorded with source, destination, protocol, and credentials used. This phase is the sole producer of L3 evidence and MHR_k findings at $k \geq 2$ by construction.

Phase 6: Network report. Phase 6 produces a network-scoped report with device inventory, per-device vulnerabilities, attack surface summary, multi-hop intrusion narrative, and recommendations ordered by severity $\times$ exposure. Cross-device aggregation is deliberately deferred to this phase to avoid bloating earlier agents, and the consolidated output enables a human analyst to prioritize remediation across the full network rather than per-host.

5.2. Cost Envelope

For a network with n devices and v findings per device on average, the harness issues $O(nv)$ LLM invocations:

$$\underbrace{1}_{P1} + \underbrace{1}_{P2} + \underbrace{n}_{P3} + \underbrace{nv}_{P4} + \underbrace{1}_{P5} + \underbrace{1}_{P6}$$

Each invocation is bounded by a fixed turn budget (50 for P1–P2, 10 per device for P3, 10 per finding for P4, 30 for P5, 20 for P6). Cost scales linearly with network size, not combinatorially.

The three structural choices above (topology prior P1, per-device parallelism P3–P4, and the dedicated intrusion phase P5) each define a distinct architectural contribution evaluated in §7.

6. Experimental Setup

Infrastructure. All 15 scenarios are deployed on a single Proxmox VE host as LXC containers on a dedicated Linux bridge with an OpenWrt router as gateway; per-scenario deployment and teardown are fully automated via Ansible. This study evaluates 12 of the 13 vulnerability-injected scenarios (`s_1`–`s_12`); `s_13` (real-VLAN Proxmox bridge) is defined in the benchmark but not included in the reported runs.

Model. To isolate architectural contributions from model-specific advantages, all LLM-driven systems run on the same general-purpose model: **MiniMax-M2.7** [42]. Security-fine-tuned models are intentionally excluded; this study addresses the architecture axis, not the model fine-tuning axis.

Baselines. We compare LANCE against three adapted LLM-agent baselines, each reimplementing a published pipeline architecture on the same scenarios and model:

- **CAI-style adapter**: single-agent tool-calling loop, adapted from the CAI [9] architecture.
- **PentGPT-style adapter**: three-phase decomposition (recon, scanning, exploitation), adapted from PentestGPT [3].
- **VulnBot-style adapter**: multi-agent with a Penetration Task Graph, adapted from VulnBot [6].

These systems are designed primarily for single-host or single-target workflows, not for full-network graph reasoning. We therefore run each adapted baseline once per ground-truth target device in a scenario, normalize the per-device outputs into the common finding schema, merge them at scenario level, and score the merged output with the same evaluator as LANCE. All three baselines use the same MiniMax-M2.7 model.

Metrics. We report recall (task completion), precision, F1, and CVSS-weighted score, formally defined in §4.5, together with cost (tokens, USD, wall-clock time). MHR_k and evidence-depth (L1/L2/L3), also defined in §4.5, are

architectural contributions of the benchmark; their systematic measurement across all scenarios is left for future work as per-hop topology annotation tooling matures.

Variance. All reported results are single-run estimates per (system, scenario) cell. The evaluator infrastructure supports repeated runs with bootstrap confidence intervals and Mann–Whitney U tests; multi-run evaluation is deferred to future work.

7. Evaluation

The evaluation is organized around three questions. First, does LANCE improve per-vulnerability scoring on the 10 network-scale IoT comparison scenarios? Second, is the gain consistent across topology types, rather than concentrated in one easy scenario? Third, how does the same harness transfer to established single-host corpora? Cross-corpus results are used only for context: models, success oracles, and task definitions differ across papers.

7.1. Overall performance

Table 4 is the controlled comparison on the 10 IoTChainBench scenarios for which all adapted baselines were run. It reports per-scenario F1 for LANCE and for each baseline architecture, all using the same MiniMax-M2.7 model and the same evaluator. Since the comparison systems are single-target agents rather than multi-device network harnesses, we run each baseline once per target device, merge its per-device findings at scenario level, and then compute the F1 shown in the table. Over the same scenarios, LANCE also reaches 0.962 recall, 0.922 precision, and an 86.1% CVSS-weighted score.

TABLE 4. PER-SCENARIO F1 ON THE 10 IOTCHAINBENCH COMPARISON SCENARIOS (MINIMAX-M2.7).

Scenario	LANCE	CAI	PentGPT	VulnBot
S1	0.923	0.150	0.000	0.150
S2	0.923	0.350	0.140	0.450
S3	0.952	0.000	0.000	0.000
S4	0.944	0.360	0.420	0.440
S5	1.000	0.500	0.640	0.500
S6	1.000	0.400	0.520	0.400
S7	0.965	0.500	0.450	0.450
S8	0.933	0.550	0.350	0.350
S9	0.952	0.170	0.430	0.290
S10	0.815	0.140	0.270	0.140
Mean	0.940	0.312	0.321	0.316

Two points stand out. First, the gap is consistent: LANCE exceeds all three adapted baselines in every comparison scenario, including both the easier star topologies and the harder segmented or role-variant cases. Second, the baselines are not empty detectors. They sometimes recover part of the injected ground truth, but their coverage is shallow enough to keep F1 low.

Figure 2 collapses the table into the headline macro-average. LANCE reaches 0.940 mean F1, while CAI, PentGPT, and VulnBot reach 0.312, 0.321, and 0.316 respectively. The aggregate gap is therefore consistent with the per-scenario view: the result is not explained by a single favorable topology.

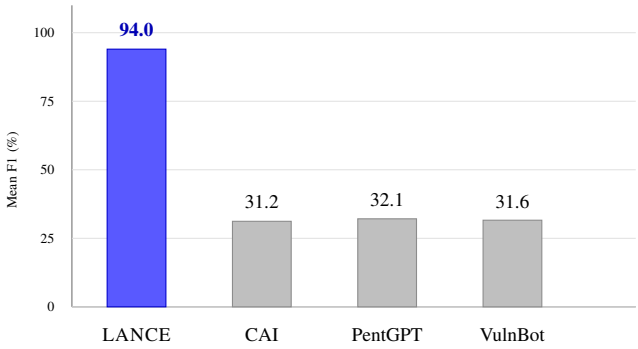

Figure 2. Macro-average F1 over the 10 comparison scenarios for all tested architectures.

7.2. Cross-benchmark comparison on single-host corpora

To assess whether LANCE generalises beyond IoT multi-hop scenarios, we run it on two single-host corpora used by prior work. Comparisons are contextual rather than head-to-head: published results differ in model, run count, and success oracle. We therefore separate the two cases: AutoPenBench has a flag-based E2E metric that supports a compact comparison table, whereas Vulhub is better reported as run-level useful-finding evidence. These corpora do not measure the full network-native architecture: they mainly stress the single-target exploitation component used inside the harness.

AutoPenBench. AutoPenBench defines 33 flag-based tasks with published E2E rates, making it the most directly comparable transfer check. We run two context policies: *blind* mode, where the agent sees only the target endpoint and exposed service metadata, and *informed* mode, where the agent also receives the case identifier and vulnerability label. Table 5 places these runs alongside published systems.

Blind mode captures 7/33 flags (21.2%) and produces useful findings on 13/33 tasks (39.4%); informed mode reaches 9/33 flags (27.3%) and 17/33 useful findings (51.5%). The blind E2E rate matches the published GPT-4o baseline (21.2%), indicating that the harness transfers to single-host exploitation without task-specific tuning. The gap with purpose-built systems—VulnBot at 30.3%, xOffense at 72.7% with a fine-tuned model—is expected: our harness was designed for multi-hop IoT networks, not single-host exploit chaining.

Vulhub. Vulhub provides reproducible vulnerable environments but no uniform flag oracle, so we report useful-finding lower bounds rather than E2E task completion. Because each

TABLE 5. AUTOPENBENCH TRANSFER RESULTS ON 33 TASKS.

System	Model	Sub-task	E2E
Llama baseline [6], [11]	Llama 3.1-405B	49.1	9.1
GPT-4o baseline [6], [11]	GPT-4o	—	21.2
VulnBot [6]	Llama 3.1-405B	69.1	30.3
xOffense [11]	Qwen3-32B LoRA	79.2¶	72.7
LANCE blind	MiniMax-M2.7	39.4†	21.2
LANCE informed	MiniMax-M2.7	51.5†	27.3

Models and run protocols differ, so this is not a controlled ranking. "—" indicates a metric not reported by the original paper. † Our sub-task analogue is useful findings*: tasks where the agent produced actionable exploit evidence, regardless of flag capture. Our harness does not use the upstream AutoPenBench orchestrator. ¶ xOffense's sub-task score is best-of-five; its E2E score is strict task completion.*

Vulhub case is a single Docker Compose target, it removes the topology prior, per-device decomposition, and lateral-movement phases that distinguish LANCE on IoTChain-Bench. The Vulhub results therefore evaluate the harness's single-target exploitation component, not the full multi-device architecture. Prior Vulhub evaluations use different target sets and oracles, so we treat them as architecture context rather than a controlled leaderboard. PentestAgent reports 74.2% full-process success on 67 VulHub targets using a multi-agent RAG pipeline [8]. CurriculumPT reports exploit success rates of 24.0% for AutoPT, 36.0% for VulnBot, 42.0% for PentestAgent, and 60.0% for its curriculum-guided multi-agent system on a 15-task Vulhub hold-out with GPT-4o-mini [17]. CheckMate samples 120 anonymized Vulhub containers and reports a $+20\%$ relative gain over Claude Code + Sonnet 4.5 [16]. Table 6 summarizes the two final Vulhub runs available for this study. They cover *different* case sets and are complementary in scope, not a blind/informed ablation.

TABLE 6. VULHUB TRANSFER RUNS.

Setting	Cases	Completed	Useful findings	Strict
Informed	191	175	69 auto; 84 reviewed	6
Blind	328	306	90 reported	6

Informed mode receives the case identifier and vulnerability label. The blind run is a distinct larger CVE superset, not the same cases without hints. "Strict" counts automatically labelled confirmed exploits. Costs/tokens are $60.96/194.5M for informed and $97.29/311.9M for blind.

In the 191-case informed run, 175 cases completed (11 failed during environment setup, 3 hit provider rate limits, 2 tripped the rate-limit breaker). The automatic verdict marks 69/175 as useful findings (39%); manual review of `proof.json` identifies 15 additional cases where the agent held exploit evidence but self-reported NO FINDING, raising the conservative lower bound to 84/175 (48%). In the 328-case blind run on a distinct superset of CVE environments, 306 cases completed, 18 failed during agent execution, and 4 failed during environment setup. This blind run was executed in two batches: 124 cases skipped after the first rate-limit breaker were replayed once the API quota recovered. The final aggregate reports useful findings on

TABLE 7. VULHUB INFORMED-RUN OUTCOME DISTRIBUTION. PERCENTAGES ARE OVER THE 175 COMPLETED AGENT RUNS UNLESS MARKED AS PRE-AGENT FAILURES.

Outcome	Cases	Share	Interpretation
Max turns	83	47%	loops and exploited-but-no-flag cases
No finding	23	13%	includes 15 hidden proof-level successes
Blocked: credentials	23	13%	exploit path required unavailable secret
Probable vulnerability	21	12%	actionable but unconfirmed
Blocked: tool	19	11%	missing exploit or runtime support
Confirmed exploit	6	3%	strict automatic success
Environment failed	11	–	target did not reach the agent
Rate-limited / skipped	5	–	provider-side interruption

90/306 cases (29%) and six strictly confirmed exploits. The gap between automatic verdicts and manual review underscores the difficulty of oracle design for heterogeneous Vulhub tasks. We therefore use Vulhub as evidence that the exploitation component transfers to established vulnerable services, while the architectural advantage of LANCE is evaluated on the multi-device IoTChainBench scenarios.

7.3. Per-scenario analysis

The per-scenario breakdown in Table 4 reveals a clear pattern: LANCE remains high across heterogeneous topology types, while the adapted baselines only partially cover the injected vulnerability sets. The hub-centric star topologies (`s_5`, `s_6`) are solved cleanly by LANCE: both reach F1 = 1.000, whereas the strongest adapted baseline on these scenarios reaches 0.640 and 0.520. Segmented IT/OT scenarios remain strong but not perfect (`s_4`: F1 = 0.944, `s_8`: F1 = 0.933), with residual errors caused by deeper pivots that increase the chance of duplicate or slightly misclassified findings. The edge-cloud pivot (`s_7`) stays high at F1 = 0.965, and the mesh smart-city case (`s_9`) reaches F1 = 0.952 with no false positives.

The most difficult comparison scenario is `s_10` (F1 = 0.815, Score = 71.3%). This topology is intentionally flat rather than multi-hop, so the degradation is not caused by lateral-movement reasoning. Instead, service role variants reduce the predictability of service semantics and increase the proportion of near-duplicate reports. The two scalability extensions show similar behavior: `s_11` achieves F1 = 0.875 and Score = 84.8%, and `s_12` achieves F1 = 0.942 and Score = 90.6%.

7.4. Failure modes and cost

Failure modes. On the IoTChainBench comparison scenarios, failures are accuracy failures rather than resource failures. The `s_10` scenario dominates the error profile, contributing both missed findings and near-duplicate false positives that arise from role variants. The remaining nine comparison scenarios each stay above F1 = 0.923, suggesting that the pipeline generalises well across varied topologies.

On the Vulhub informed run (Table 7), the dominant outcome is MAX TURNS (47%), which conflates two behaviours: genuinely unproductive search loops and successful exploitation followed by continued searching for a flag that the Vulhub environment does not standardise. The next most actionable failure categories are tool coverage (19 cases blocked by missing exploit support) and credential handling (23 cases requiring secrets the agent did not possess). The blind run shows the same broad pattern at a larger scale: among 306 completed cases, 131 end at the turn budget, 85 report NO FINDING, 41 block on credentials, and 25 block on missing tools.

Computational cost. The 10 main IoTChainBench scenarios run at a cost that scales with network size. On the AutoPenBench single-host corpus, blind-mode evaluation costs $4.88 and consumes 15.5M tokens for 33 tasks ($0.15 per task); informed mode costs $7.43 and 23.6M tokens ($0.23 per task). The Vulhub informed run (191 cases, six workers) consumed 194.5M tokens at a total cost of $60.96 ($0.33 per completed case). The larger blind run consumed 311.9M tokens at $97.29, or approximately $0.32 per completed case. These figures suggest that LANCE remains within a practical per-run budget for research deployments.

8. Discussion and Limitations

Where LLMs help, where they don't. IoTChainBench is designed to separate failure modes that binary flag-capture benchmarks collapse into a single signal. The results show strong aggregate vulnerability matching across varied topologies, with the per-scenario heatmap confirming that the gain is not driven by a single easy pattern. The intended deployment model is hybrid: LLM agents for configuration auditing and multi-step reasoning, traditional scanners for broad CVE coverage, and human operators or specialised tooling for the OT/ICS layer.

Implications for agentic systems. The harness's per-device and per-vulnerability micro-agent structure is one answer to context scaling, but it cedes cross-device inference. Designs that keep a global aggregator agent in the loop without paying the full network-context cost are an open research direction.

Limitations. (i) *Simulated infrastructure.* IoTChainBench scenarios emulate IoT devices on Linux VMs; real constrained hardware introduces embedded-specific attack surfaces (bootloaders, RTOS, firmware) we do not cover. (ii) *Evaluator leniency.* Bonus categories are accepted as non-FP even without ground-truth matches, which inflates precision. (iii) *Model variance.* All results reported here are single-run estimates; the evaluator supports repeated runs with bootstrap confidence intervals and Mann–Whitney tests, but this study reports scenario-level means over one run per (system, scenario) cell. (iv) *Prompt sensitivity.* We fix a single prompt family across models; per-model prompt tuning would likely raise scores. (v) *Static architectures.* The five patterns are fixed; real deployments evolve over time and include devices not represented here (RFID badge readers, industrial PLCs, medical devices).

9. Ethical Considerations

Isolation. All experiments run on an isolated Proxmox cluster with a dedicated bridge (`vmbr1`) and no route to external networks; the OpenWrt gateway's `WAN` interface is firewall-isolated. No live hosts, production services, or third-party infrastructure are probed.

Responsible disclosure. IoTChainBench vulnerabilities are deterministically injected into our own lab VMs via Ansible. No 0-days are introduced; all categories (misconfig, weak credentials, Terrapin CVE-2023-48795, outdated Dropbear) correspond to patterns already publicly documented. No responsible-disclosure process is triggered.

Misuse risk of released artifact. The benchmark artifact (Ansible playbooks, ground truth, evaluator) does not embed exploitation payloads for unknown vulnerabilities, and the injected weaknesses are common misconfigurations documented in OWASP IoT Top 10. The harness uses only standard pentest tools already in widespread use. We judge the net societal benefit (reproducible evaluation of LLM pentest agents, and a concrete evaluation surface for future defenders) to exceed the incremental misuse risk.

IRB / human subjects. No human subjects are involved. No personal data is collected or processed.

LLM costs and environmental footprint. We report token-estimated cost, actual API billing, and wall-clock time with the experimental results. Energy usage is not claimed unless measured from provider- or host-side telemetry.

10. Conclusion

We presented IoTChainBench, a reproducible benchmark for evaluating LLM penetration testing agents at *network scale* on IoT infrastructure, and LANCE, a network-native multi-agent harness whose infrastructure-graph guidance and dedicated lateral movement phase keep LLM context tractable while explicitly modelling reachability. LANCE achieves 0.940 mean F1 and 86.1% weighted score on the 10 main scenarios, outperforming each adapted LLM-agent baseline in every scenario. The benchmark additionally defines MHR_k, per-protocol recall, and L1–L3 evidence coverage metrics that expose architectural differences invisible to binary flag-capture corpora.

Future work. (i) *Multi-model robustness:* extending the architectural comparison to recent frontier and open-weight models to confirm model-independence of the architectural advantage; (ii) *Real hardware:* extending to embedded Zigbee, LoRaWAN, and sub-GHz attack surfaces via SDR tooling; (iii) *Defender's side:* using IoTChainBench as a benchmark for LLM-assisted IoT defense (detection, patching recommendation).

All code, scenarios, ground truth, and run logs are released at `ANONYMIZED_URL` under an open license.

References

[1] A. Happe and J. Cito, "Getting pwn'd by AI: Penetration testing with large language models," in *Proc. ACM ESEC/FSE*, 2023.

[2] A. Happe *et al.*, "HackingBuddyGPT: An LLM-driven pentest agent," 2024. [Online]. Available: https://github.com/ipa-lab/hackingBuddyGPT

[3] G. Deng *et al.*, "PentestGPT: Evaluating and harnessing large language models for automated penetration testing," in *Proc. 33rd USENIX Security Symp.*, 2024.

[4] J. Xu *et al.*, "AutoAttacker: A large language model guided system to implement automatic cyber-attacks," *arXiv:2403.01038*, 2024.

[5] O. V. Aguinaga, "Nebula: An LLM-powered penetration testing assistant," *arXiv:2412.00006*, 2024.

[6] H. He *et al.*, "VulnBot: Autonomous penetration testing for a multi-agent collaborative framework," *arXiv:2501.13411*, 2025.

[7] Y. Ginige *et al.*, "AutoPentester: An LLM agent-based framework for automated pentesting," *arXiv:2510.05605*, 2025.

[8] X. Shen *et al.*, "PentestAgent: Incorporating LLM agents to automated penetration testing," in *Proc. 20th ACM Asia Conf. Computer and Communications Security (ASIA CCS)*, 2025. doi: 10.1145/3708821.3733882.

[9] V. Mayoral-Vilches *et al.*, "CAI: An open, bug bounty-ready cybersecurity AI," *arXiv:2504.06017*, 2025.

[10] Anonymous, "Pentest-R1: Towards autonomous penetration testing reasoning optimized via two-stage reinforcement learning," *arXiv:2508.07382*, 2025.

[11] P. D. Luong *et al.*, "xOffense: An autonomous multi-agent framework for penetration testing with domain-adapted large language models," *arXiv:2509.13021*, 2025.

[12] T. Abramovich *et al.*, "EnIGMA: Interactive tools substantially assist LM agents in solving CTF challenges," in *Proc. ICML*, 2025.

[13] Z. Li, S. Dutta, and M. Naik, "IRIS: LLM-assisted static analysis for detecting security vulnerabilities," in *Proc. NDSS*, 2025.

[14] Anonymous, "Context relay for long-running penetration-testing agents," in *NDSS Workshop on LLM-Assisted Security and Trust Exploration (LAST-X)*, 2026.

[15] Anonymous, "AWE: A memory-augmented multi-agent framework for autonomous web penetration testing," in *NDSS Workshop on LLM-Assisted Security and Trust Exploration (LAST-X)*, 2026.

[16] L. Wang, X. Shi, Z. Li, Y. Jiang, S. Tan, Y. Jiang, J. Cheng, W. Chen, X. Shen, Z. Li, and Y. Chen, "Automated penetration testing with LLM agents and classical planning," *arXiv:2512.11143*, 2025.

[17] X. Wu, Y. Tian, Y. Chen, P. Ye, X. Cui, J. Jia, S. Li, J. Liu, and W. Niu, "CurriculumPT: LLM-based multi-agent autonomous penetration testing with curriculum-guided task scheduling," *Applied Sciences*, vol. 15, no. 16, Art. 9096, 2025.

[18] L. Gioacchini, M. Mellia, I. Drago, A. Delsanto, G. Siracusano, and R. Bifulco, "AutoPenBench: Benchmarking generative agents for penetration testing," *arXiv:2410.03225*, 2024.

[19] M. Shao *et al.*, "NYU CTF Bench: A scalable open-source benchmark dataset for evaluating LLMs in offensive security," in *Proc. NeurIPS Datasets and Benchmarks Track*, 2024.

[20] A. K. Zhang *et al.*, "Cybench: A framework for evaluating cybersecurity capabilities and risk of language models," *arXiv:2408.08926*, 2024.

[21] M. Bhatt *et al.*, "CyberSecEval 2: A wide-ranging cybersecurity evaluation suite for large language models," *arXiv:2404.13161*, 2024.

[22] V. Mayoral-Vilches *et al.*, "CAIBench: A meta-benchmark for evaluating cybersecurity AI agents," *arXiv:2510.24317*, 2025.

[23] Ludus Cloud, "Ludus: Cyber range automation on Proxmox." [Online]. Available: https://ludus.cloud

[24] M3, "GOAD: Game of Active Directory." [Online]. Available: https://github.com/Orange-Cyberdefense/GOAD

[25] Vulhub contributors, "Vulhub: Pre-built vulnerable Docker environments for learning and reproducing CVEs." [Online]. Available: https://github.com/vulhub/vulhub

[26] M. Antonakakis *et al.*, "Understanding the Mirai botnet," in *Proc. 26th USENIX Security Symp.*, 2017.

[27] W. Turton, "Hackers breach thousands of security cameras, exposing Tesla, jails, hospitals," *Bloomberg*, Mar. 2021.

[28] R. Langner, "Stuxnet: Dissecting a cyberwarfare weapon," *IEEE Security & Privacy*, vol. 9, no. 3, pp. 49–51, 2011.

[29] C. Phillips and L. P. Swiler, "A graph-based system for network-vulnerability analysis," in *Proc. New Security Paradigms Workshop*, 1998.

[30] O. Sheyner, J. Haines, S. Jha, R. Lippmann, and J. M. Wing, "Automated generation and analysis of attack graphs," in *Proc. IEEE Symp. Security and Privacy*, 2002.

[31] X. Ou, S. Govindavajhala, and A. W. Appel, "MulVAL: A logic-based network security analyzer," in *Proc. USENIX Security Symp.*, 2005.

[32] OWASP Foundation, "OWASP Internet of Things Top 10," 2024. [Online]. Available: https://owasp.org/www-project-internet-of-things-top-10/

[33] MITRE Corporation, "MITRE ATT&CK for ICS," 2024. [Online]. Available: https://attack.mitre.org/matrices/ics/

[34] K. Stouffer *et al.*, "Guide to operational technology security," National Institute of Standards and Technology, NIST SP 800-82 Rev. 3, 2023.

[35] S. Garcia, A. Parmisano, and M. J. Erquiaga, "IoT-23: A labeled dataset with malicious and benign IoT network traffic," Stratosphere Laboratory, 2020. [Online]. Available: https://www.stratosphereips.org/datasets-iot23

[36] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, and A. Anwar, "TON_IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-driven intrusion detection systems," *IEEE Access*, vol. 8, pp. 165130–165150, 2020.

[37] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, "Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning," *IEEE Access*, vol. 10, pp. 40281–40306, 2022.

[38] A. Costin, J. Zaddach, A. Francillon, and D. Balzarotti, "A large-scale analysis of the security of embedded firmwares," in *Proc. USENIX Security Symp.*, 2014.

[39] D. D. Chen, M. Egele, M. Woo, and D. Brumley, "Towards automated dynamic analysis for Linux-based embedded firmware," in *Proc. NDSS*, 2016.

[40] M. Kim, D. Kim, E. Kim, S. Kim, Y. Jang, and Y. Kim, "FirmAE: Towards large-scale emulation of IoT firmware for dynamic analysis," in *Proc. Annual Computer Security Applications Conf. (ACSAC)*, 2020.

[41] OWASP, "IoTGoat: A deliberately insecure IoT firmware," 2024. [Online]. Available: https://github.com/OWASP/IoTGoat

[42] MiniMax, "MiniMax-M2.7: A general-purpose large language model," 2026. [Online]. Available: https://www.minimax.io/models/text/m27

LLM Usage Statement

LLMs were used for editorial assistance while preparing this manuscript, and all generated text was inspected by the authors to ensure accuracy, originality, and consistency with

the underlying artifacts. LLMs are also part of the studied methodology: LANCE uses tool-calling LLM agents for network reconnaissance, vulnerability analysis, exploit verification, intrusion attempts, and report generation, as described in §5, §6, and §7.